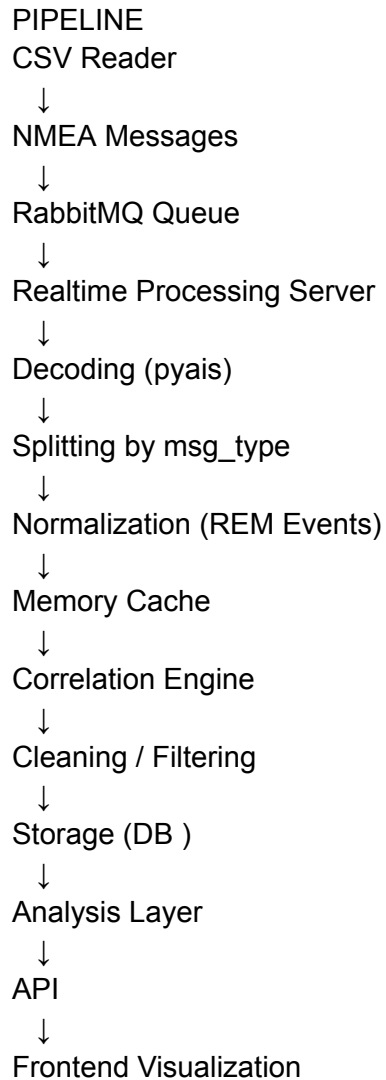




FOLDER STRUCTURE

AIS folder

-> CSV Reader folder

-> csv_to_queue.py

-> input folder

-> AIS_Klaipeda_From20250908_To20251008 2.csv

-> Realtime Pipeline folder

-> realtime_pipeline.py

-> outputs folder

Decoded and cleaned files will be generated here as csv for now later on stored in DB.

STEPS TO RUN RABBITMQ

In the AIS folder:

Execute following commands

- brew install rabbitmq
- brew services start rabbitmq
- rabbitmqctl status
- Open on browser : <http://localhost:15672>
Username: guest
Password: guest

Run on TERMINAL 1:

python3 csv_to_queue.py

On TERMINAL 2:

python3 realtime_pipeline.py

- Csv reader (Producer) sends nmea sentences -> Queue - > Server (consumer)
- On server side - decodes the message, splitting according to message types, normalization according to scheme , applying cleaning filters -> storing in csv

Realtime Pipeline ([realtime_pipeline.py](#)) Overview

1. Connects to RabbitMQ input queue
 - Listens for incoming AIS NMEA messages sent from [csv_to_queue.py](#) or a live AIS source.
2. Decodes NMEA messages
 - Uses [pyais](#) to convert raw NMEA sentences into structured Python dictionaries.
 - Adds a timestamp immediately after decoding to preserve message sequence for correlation -> [REM events](#)
3. Categorizes messages by type
 - Uses [MESSAGE_MAP](#) to assign each message to one of these categories:
 - [dynamic_position](#) (Class A moving ships)
 - [static_position](#) (Class B moving ships)
 - [voyage_info](#) (static voyage information)
 - [base_station](#) (shore stations)

- `navigation_aid` (buoys, lighthouses)
 - `safety_messages` (all safety-related messages)
 - `binary_misc` (everything else)
4. Cleans data per category
 - Applies validation rules for MMSI, latitude, longitude, speed, etc.
 - Removes invalid or corrupt messages before storage.
 5. Stores cleaned messages in CSV
 - Writes each category to its own CSV file in `outputs/` folder for inspection and further processing.
 6. Generates REM (Real Event Messages)
 - For `dynamic_position`, `static_position`, and `safety_messages`.
 - Adds a timestamped event row in `rem/rem_events.csv` for later correlation/analysis.
 7. Sends cleaned messages to an output RabbitMQ queue
 - Allows downstream components (e.g., storage, memory cache, correlation engine) to consume already cleaned and categorized messages.
 8. Handles errors gracefully
 - Prints problematic NMEA sentences but continues processing the rest of the stream.
 - Ensures CSV files and REM files are flushed and closed properly on shutdown.